Assignment 3: Building Blocks of GAN

Question 1

For a transposed convolution with dilation 1, each spatial output dimension is:

$O = (I - 1)S - 2P + (K - 1) + OP + 1$
$= (I - 1)S - 2P + K + OP.$

Substitute $I = 8$, $K = 4$, $S = 2$, $P = 1$, $OP = 1$:

$O = (8 - 1) \times 2 - 2 \times 1 + 4 + 1$
$= 14 - 2 + 4 + 1 = 17.$

The spatial output is **17 x 17**.

Question 2

Keeping the values in Question 1 fixed except for $S = 3$:

$O = (8 - 1) \times 3 - 2 \times 1 + 4 + 1$
$= 21 - 2 + 4 + 1 = 24.$

The output changes from **17 x 17 to 24 x 24**. In general, increasing stride from 2 to 3 increases each output dimension by $I - 1$ (7 in this example).

Question 3

With dilation 1:

$O = (I - 1)S - 2P + K + OP.$

Apply this separately to height and width. More generally, with dilation D:

$O = (I - 1)S - 2P + D(K - 1) + OP + 1.$

Here I is input size, K kernel size, S stride, P padding and OP output padding.

Question 4

Choose **kernel size 2 x 2, stride 2, padding 0, output padding 0**, dilation 1:

$O = (16 - 1) \times 2 - 0 + 2 + 0 = 30 + 2 = 32.$

Thus a 16 x 16 map becomes 32 x 32. A 4 x 4 kernel with stride 2 and no padding would give 34 x 34, so it does not meet this question's condition.

Question 5

The mini-batch is [6, 8, 10, 6], with N = 4.

Mean: mu = (6 + 8 + 10 + 6) / 4 = 30 / 4 = 7.5.
Deviations: [-1.5, 0.5, 2.5, -1.5].
Squared deviations: [2.25, 0.25, 6.25, 2.25].
Batch variance: var = (2.25 + 0.25 + 6.25 + 2.25) / 4 = 11 / 4 = 2.75.
Standard deviation: sqrt(2.75) = sqrt(11) / 2, approximately 1.658312.

Without scale gamma or shift beta, normalized values are (x - mu) / sqrt(var):

[-3/sqrt(11), 1/sqrt(11), 5/sqrt(11), -3/sqrt(11)]

Approximately **[-0.904534, 0.301511, 1.507557, -0.904534]**.

Their mean is 0 and their variance is 1. BatchNorm uses the population-style variance (divide by N) for training-time normalization, not the sample variance (divide by N - 1). Real implementations add epsilon inside the square root: (x - mu)/sqrt(var + epsilon), so the resulting variance is slightly below 1. The values above use epsilon = 0 as the question requests unit variance.

Question 6

ReLU(x) = max(0, x): x for x >= 0, and 0 for x < 0.

LeakyReLU(x) = x for x >= 0, and alpha*x for x < 0, where 0 < alpha < 1.
For this assignment, alpha = 0.2.

The key difference is the negative-input slope: ReLU has derivative 0 there, whereas LeakyReLU has derivative alpha. Both have positive-input slope 1. At x = 0 they have a kink; implementations choose a derivative convention.

Question 7

A ReLU neuron whose pre-activation stays negative produces zero output and zero local gradient. Its incoming parameters may therefore stop receiving a useful learning signal (the dying-ReLU problem).

LeakyReLU keeps a small nonzero slope on the negative side, allowing gradients to propagate and making such units less likely to become permanently inactive. This can help optimization in deep networks and GAN discriminators. It does not guarantee better results for every network or dataset.

Prepared with OpenAI Codex assistance. Review and understand the derivations and the code before submission.